Prescriptive Assignment 3 Report

Prediction & Predictive coming together

Rick de Rijk, Tycho van Rooij, Stefan Vonk & Garoe

Table of contents

1 Introduction **2**

2 Prediction Task **2**
2.1 Data Analysis and Feature Engineering . . . 2
2.2 Model Development . . . 2
2.3 Prediction Results . . . 3
2.4 Comparison with Consultancy Predictions . . . 4

3 Optimization Task **4**
3.1 Problem Formulation . . . 4
3.2 Genetic Algorithm Design . . . 4
3.2.1 Representation . . . 4
3.2.2 Selection Strategy . . . 4
3.2.3 Crossover . . . 5
3.2.4 Mutation . . . 5
3.2.5 Feasibility Handling . . . 5
3.2.6 Hyperparameters . . . 5
3.3 Optimization Results . . . 5
3.4 Performance Analysis . . . 5
3.5 Comparison with Consultancy RUL . . . 5

4 Discussion **5**

5 Conclusion **6**

6 References **6**

7 Appendix **6**

List of Figures

1 variance Bar chart . . . 6
2 flat-sensors . . . 6
3 Sensor trends VS RUL . . . 7
4 Pearson Correlation . . . 7
5 Heatmap . . . 8
6 Predicted vs actual + residuals . . . 8
7 Histogram-scatter-boxplot . . . 9

1 Introduction

Company X operates a fleet of 100 aircraft engines subject to wear and tear over time. To avoid unsafe operation and minimise costs, engines must be maintained before they fail. A key quantity for planning this maintenance is the Remaining Useful Lifetime (RUL): the number of cycles an engine can still operate before failure. This report addresses two tasks: first, developing a predictive model to estimate the RUL of engines currently in service; second, using those predictions as input to a Genetic Algorithm that constructs an optimal maintenance schedule. This report covers the Prediction Task in full.

2 Prediction Task

2.1 Data Analysis and Feature Engineering

Dataset. The training dataset (DataTrain) contains run-to-failure observations for 100 engines across 20,631 rows, with 21 sensor measurements and 3 operational settings recorded per cycle. Engines begin with varying degrees of initial wear and are recorded until failure.

Constructing RUL. The dataset does not include an explicit RUL column. Since each engine's final recorded cycle corresponds to failure, the RUL at any earlier cycle is the number of cycles remaining until that point. This yields:

```
RUL = max_cycle(engine_id) - cycle
```

Applied row-wise using a grouped transform, this produces RUL values ranging from 0 (failure) to 361 cycles, with a mean of approximately 108 cycles. The cycle index itself is not used as a model feature: an engine at cycle 50 may have very different RUL depending on its total lifespan, making the raw cycle value non-transferable across engines.

Exploratory Data Analysis. Variance was computed across all sensors and settings to identify informative features. Ten sensors and all three operational settings exhibited near-zero variance (< 0.01), meaning their values remained essentially constant across the entire engine lifecycle and carry no predictive signal Figure 1 . The flat line behaviour of these sensors is illustrated in Figure 2. These were flagged for removal. The remaining sensors were plotted against RUL for a sample of engines, confirming systematic monotonic trends: sensors such as `sensor_val19` and `sensor_val10` increase as RUL decreases, while `sensor_val2` and `sensor_val5` decrease Confirmed by plotting sensor values against RUL for sample of three engines Figure 3. Pearson correlation analysis confirmed these observations, with six sensors exceeding $|r| = 0.5$ with RUL Figure 4. A correlation heatmap further revealed substantial multicollinearity among retained sensors, notably `sensor_val1` and `sensor_val3` ($r = 0.96$) and `sensor_val2` and `sensor_val5` ($r = 0.81$), indicating redundancy in the raw feature set Figure 5.

Feature Engineering. Based on the EDA, the following decisions were made. All near-constant features were dropped: `set1`, `set2`, `set3`, `sensor_val7`, `sensor_val8`, `sensor_val9`, `sensor_val11`, `sensor_val13`, `sensor_val14`, `sensor_val16`, `sensor_val18`, and `sensor_val20`. The 12 retained sensors were kept as raw features. Additionally, 10-cycle rolling means and rolling standard deviations were computed per engine for each retained sensor. The rolling mean captures the local degradation trend rather than the noisy instantaneous reading, while the rolling std captures volatility, which can itself be a degradation signal. The `min_periods=1` argument ensures no missing values are introduced for the first cycles of each engine. The final feature set consists of 36 features: 12 raw sensors, 12 rolling means, and 12 rolling standard deviations.

2.2 Model Development

Model choice. XGBoost (gradient boosted trees) was selected for its ability to handle non-linear relationships, robustness to multicollinearity between features, and strong out-of-the-box performance on tabular regression tasks without requiring feature scaling.

Data splitting. The 100 training engines were split into 80 training engines and 20 held-out test engines (80/20). The split is performed at the engine level rather than the row level to prevent data leakage: rows from the same engine are temporally correlated, so a row-level split would allow the model to observe earlier cycles of an engine during training and later cycles of the same engine at test time. The 20 test engines were never seen during training and constitute a genuine out-of-sample evaluation. The model was configured as follows:

Table 1: Hyperparameter Tuning

Hyperparameter	Value	Justification
`n_estimators`	400	Sufficient rounds to converge at the chosen learning rate
`max_depth`	4	Shallow trees reduce overfitting on a ~16,000-row training set
`learning_rate`	0.05	Conservative step size paired with 400 estimators
`subsample`	0.8	Row subsampling per tree; reduces variance between trees
`colsample_bytree`	0.8	Feature subsampling; reduces correlation between trees
`reg_alpha`	0.1	L1 regularisation; encourages sparsity
`reg_lambda`	2	L2 regularisation; shrinks leaf weights against overfitting
`min_child_weight`	3	Prevents splits on very small data subsets

The combination of row and feature subsampling implements stochastic gradient boosting, which is particularly beneficial given the multicollinearity in the feature set.

Validation. Five-fold cross-validation was applied to the 80 training engines using `GroupKFold` with `engine_id` as the grouping variable, ensuring all rows of a given engine remain within a single fold. Standard `KFold` would mix rows from the same engine across folds, reintroducing the leakage problem. Cross-validation was scored using RMSE.

Evaluation metrics. RMSE is the primary metric. It is expressed in cycles, making it directly interpretable as average prediction error, and penalises large errors more heavily, appropriate since severely underestimating RUL could result in missed maintenance and safety penalties. MAE is reported as a complementary metric, treating all errors equally and representing the typical prediction error. R^2 measures the proportion of variance in RUL explained by the model.

2.3 Prediction Results

The model was retrained on all 100 training engines before predicting on DataSchedule, maximising the available training information. The identical feature engineering pipeline was applied to DataSchedule. Since DataSchedule does not contain run-to-failure sequences, predictions were made at the last recorded cycle per engine, representing the engine's current state. All predictions were rounded to integer values and clipped at zero.

Cross-validation yielded a mean RMSE of **42.94 ± 3.30 cycles** across five folds, with individual fold scores ranging from 38.65 to 46.34. The low standard deviation confirms stable performance across different engine subsets. The held-out test set yielded an RMSE of **45.85 cycles**, MAE of **33.27 cycles**, and R^2 of **0.593**. The proximity of the test RMSE to the cross-validation mean (difference of ~3 cycles)

confirms that no overfitting occurred, validating the engine-level GroupKFold setup and regularisation choices. The model performs best at low RUL values, the operationally critical region for maintenance scheduling and shows increasing prediction error at high RUL, attributable to the relative scarcity of high-RUL observations in run-to-failure training data. The scatter and residual plots confirm these findings Figure 6

Across the 100 DataSchedule engines, predicted RUL values range from 8 to 219 cycles, with a mean of **94.9 cycles** and a standard deviation of 52.9 cycles.

2.4 Comparison with Consultancy Predictions

The consultancy's predictions cover the same 100 DataSchedule engines, with a mean of **82.9 cycles** and a standard deviation of 48.4 cycles. Our model predicts on average **12.0 cycles higher**, with an MAE between the two prediction sets of **21.45 cycles**.

Two paired statistical tests were applied. The **Wilcoxon signed-rank test** (stat = 1187.50, $p < 0.0001$) is the primary test, selected as a non-parametric method appropriate for RUL predictions which are strictly positive and right-skewed. The **paired t-test** (stat = 4.43, $p < 0.0001$) is reported as a parametric complement. Both tests confirm that the difference between the two prediction sets is statistically significant and not attributable to random variation. The three comparison plots are shown in Figure 7

The scatter plot of our predictions against the consultancy's shows broad agreement at low RUL values, with divergence increasing at higher values, consistent with our model's known overestimation tendency in that region. The histogram of pairwise differences is right-skewed with a mean of 12.0 cycles, indicating that when the two models disagree strongly, it is predominantly our model that predicts the higher value. The boxplot confirms that our model's predictions have a higher median and wider interquartile range than the consultancy's, reflecting greater spread across individual engine predictions.

The systematic positive offset is consistent with the bias observed during model evaluation and is attributable to the underrepresentation of high-RUL observations in the training data. Practically, our higher RUL predictions push safety due dates further into the future, resulting in fewer engines falling within the 30-day planning horizon compared to the consultancy's more conservative estimates. The downstream impact of this difference on the maintenance schedule is examined in the Optimization Task.

3 Optimization Task

3.1 Problem Formulation

- Description of maintenance scheduling problem
- Constraints
- Objective: minimize penalty costs

3.2 Genetic Algorithm Design

3.2.1 Representation

- Description of an individual

3.2.2 Selection Strategy

- Method used

3.2.3 Crossover

- Description of crossover method

3.2.4 Mutation

- Description of mutation method or justification

3.2.5 Feasibility Handling

- How infeasible solutions are handled

3.2.6 Hyperparameters

- Population size
- Crossover probability
- Mutation probability
- Other design choices

3.3 Optimization Results

- Best solution found
- Description of maintenance schedule
- Total penalty cost

3.4 Performance Analysis

- Description of 30 runs experiment
- Average-best-fitness plot
- Interpretation of results

3.5 Comparison with Consultancy RUL

- Results using consultancy predictions
- Comparison with own predictions
- Explanation of differences

4 Discussion

- Key insights

- Strengths of the approach
- Limitations

5 Conclusion

- Summary of findings
- Final remarks

6 References

- Sources used

7 Appendix

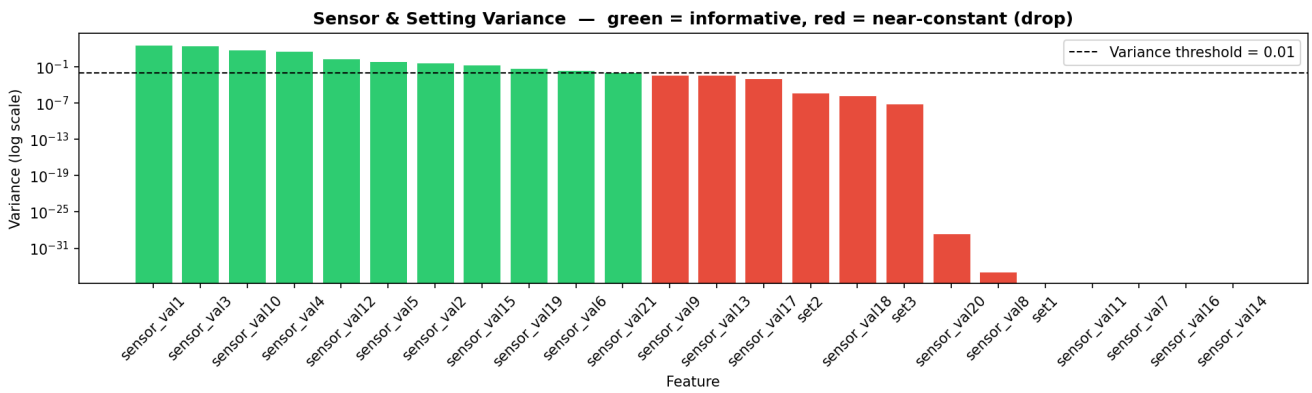

Figure 1: variance Bar chart

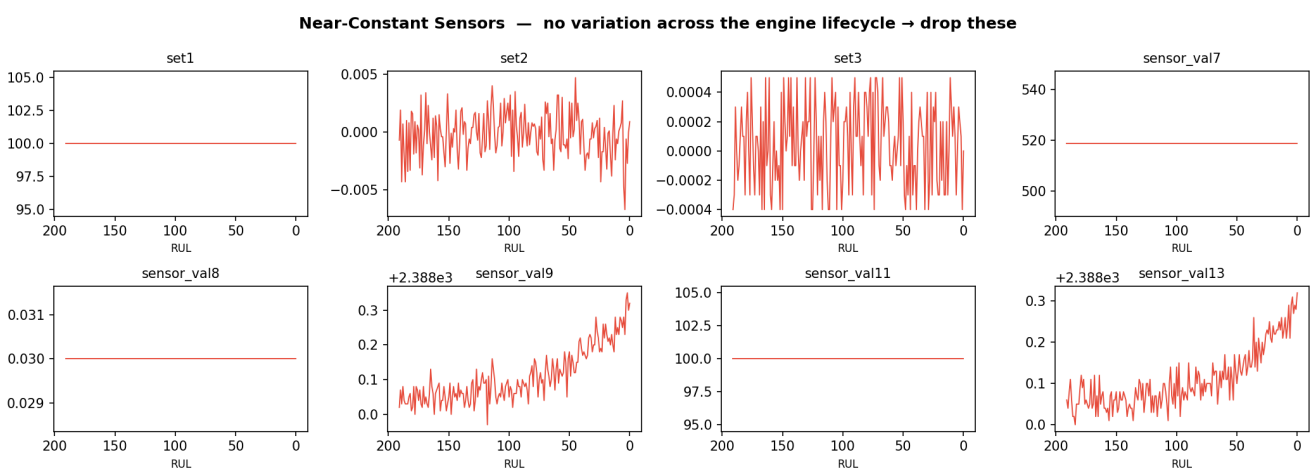

Figure 2: flat-sensors

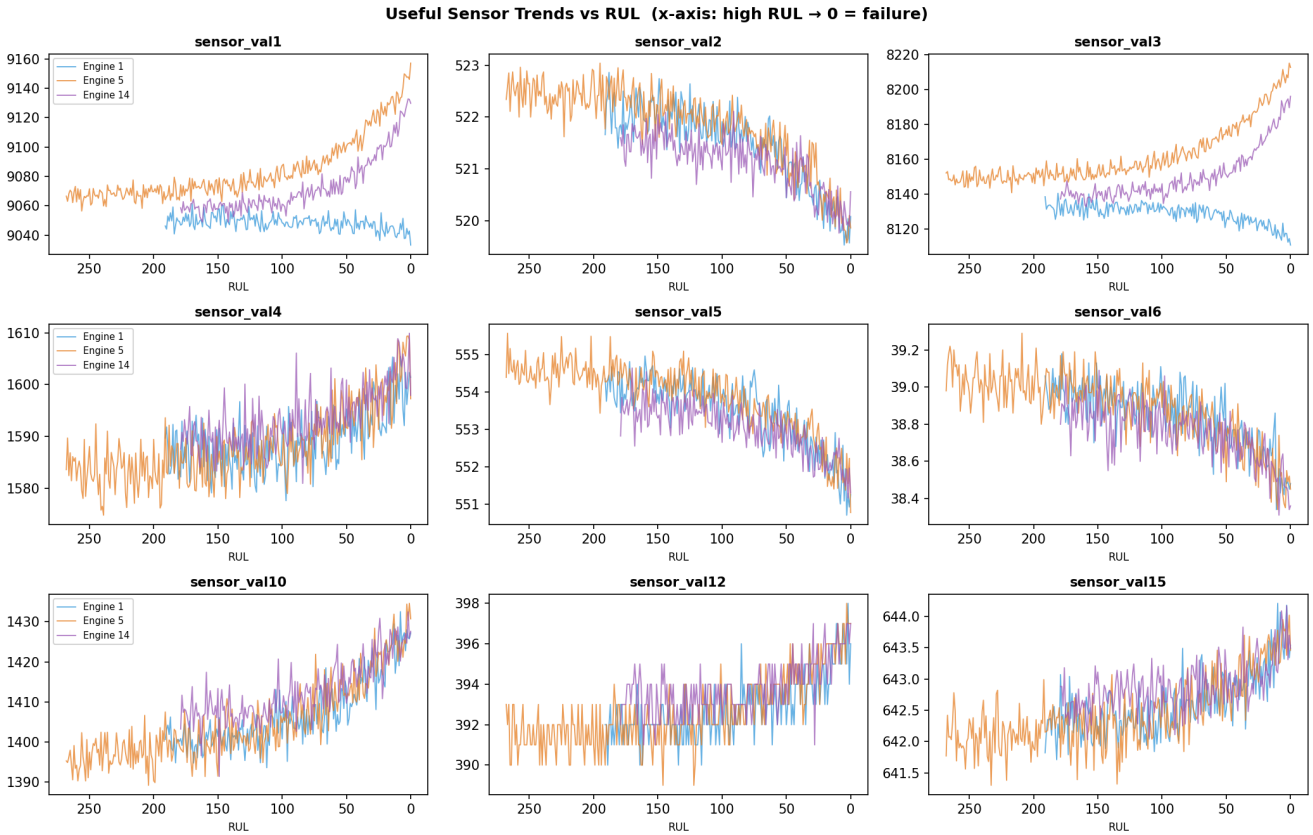

Figure 3: Sensor trends VS RUL

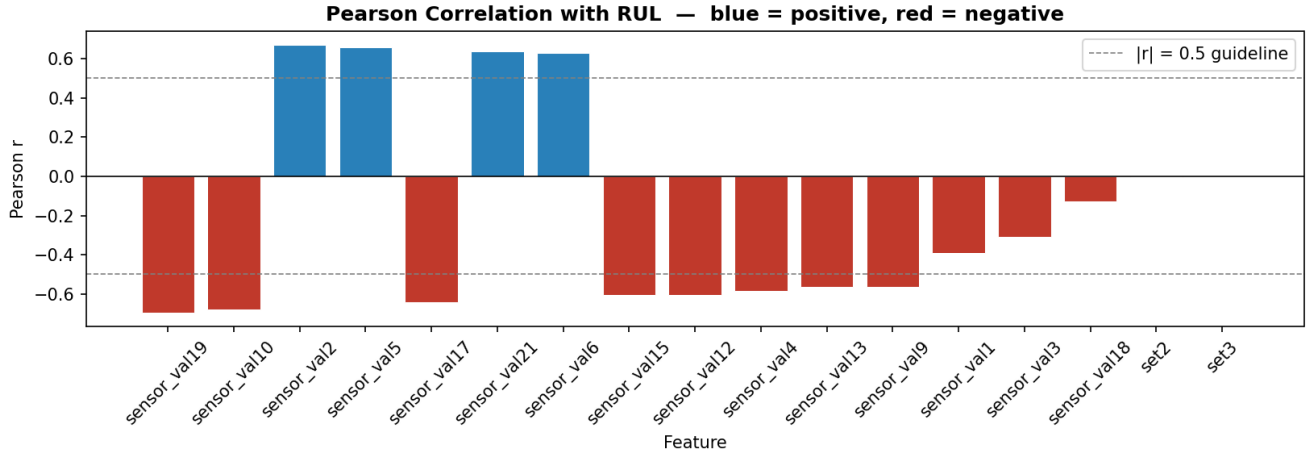

Figure 4: Pearson Correlation

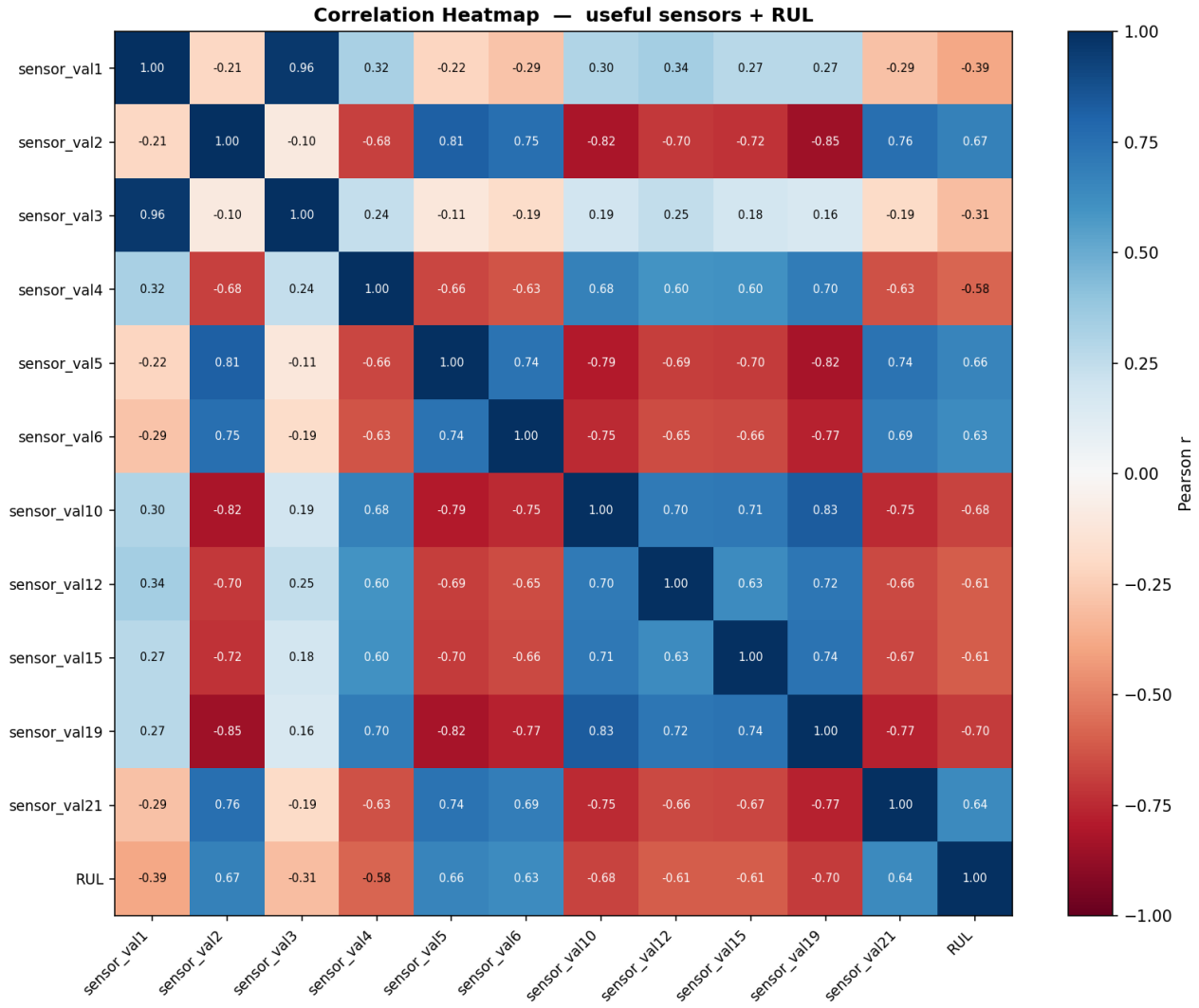

Figure 5: Heatmap

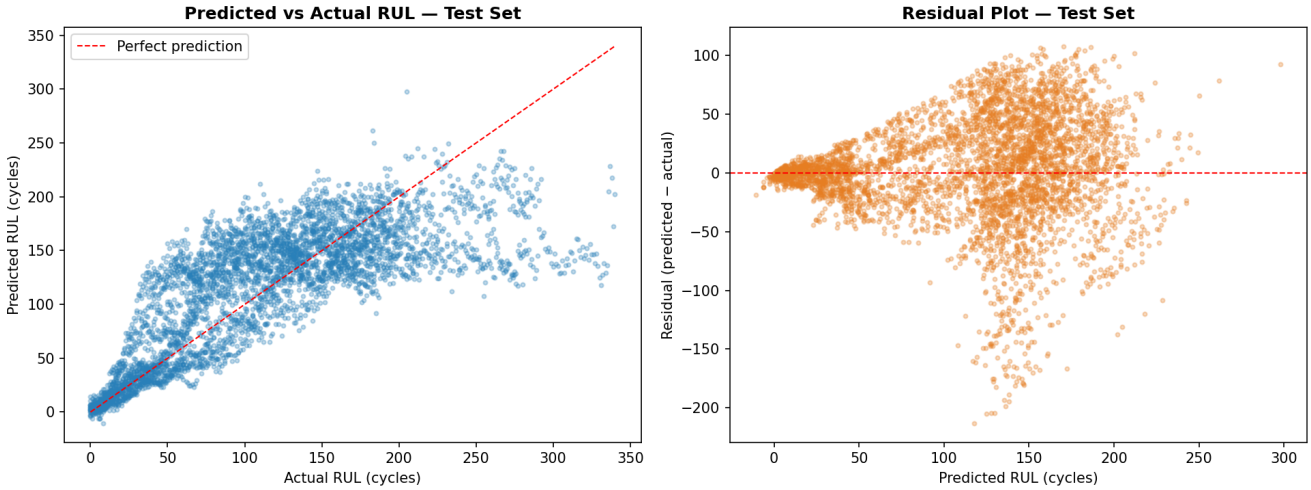

Figure 6: Predicted vs actual + residuals

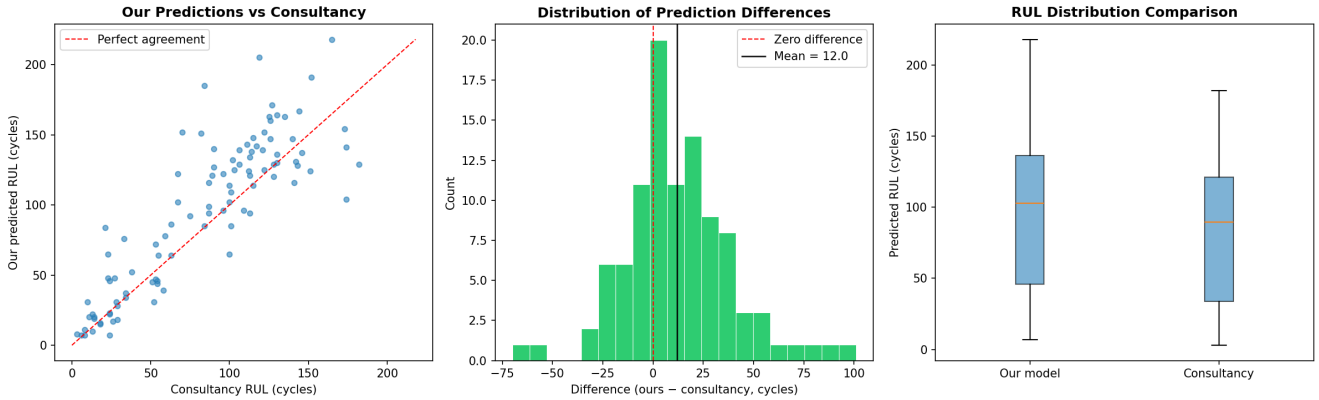

Figure 7: Histogram-scatter-boxplot

8